UNIVERSIDAD PRIVADA DEL VALLE

Base de Datos III (BDD3T4) · Avance 2 · Semanas 2 y 3

INFORME FINAL DE INTEGRACIÓN

Paquete PKG_TURISMOVIVE y trigger de resguardo TRG_RESGUARDO_RESERVAS

Grupo 6 — TurismoVive · Septiembre de 2026

Integrantes: Alejandro Rojas Espinoza · Alejandro Yabeta Arias · Mathias Ethan Cordova Rodrigues · Sebastian Maldonado Salazar

---

1. Contexto y proceso de negocio que se automatiza

TurismoVive es una agencia de turismo receptivo que vende paquetes hacia el Salar de Uyuni, La Paz y la Amazonía boliviana. Cada paquete combina actividades variables, su precio depende de un precio base, del costo de las actividades y de la comisión del agente, y su capacidad es limitada. La Pregunta Rectora pide una plataforma que permita **armar paquetes flexibles, calcular tarifas dinámicas y comisiones, mostrar la disponibilidad de cupos en tiempo real y anticipar la demanda estacional**.

El proceso central es la **venta de una reserva**: (1) el agente consulta el paquete, sus actividades y su precio; (2) verifica que queden cupos; (3) registra la reserva del viajero; (4) la confirma o la cancela, y (5) calcula el importe a cobrar. En el Avance 1 cada paso era una consulta suelta y la verificación de cupos dependía del agente, con riesgo real de **sobreventa**. Este avance automatiza el proceso completo con dos componentes: el paquete `PKG_TURISMOVIVE`, que ejecuta cada paso, y el trigger `TRG_RESGUARDO_RESERVAS`, que garantiza que la regla de cupos se cumpla aunque alguien no use el paquete.

2. Arquitectura de la solución

La solución se construye en tres niveles sobre la Capa Transaccional de Oracle. Ningún script del Avance 2 modifica tipos ni tablas del Avance 1: el paquete y el trigger se apoyan en ellos tal como fueron entregados.

Nivel	Componentes	Responsabilidad
Datos (Avance 1)	`TIPO_PAQUETE_TURISTICO` con Nested Table `ACTIVIDADES_NT`, `TABLA_VIAJEROS`, `TABLA_RESERVAS` con `REF` + `SCOPE FOR`, método `calcula_precio_total()`	Almacenar paquetes flexibles, viajeros y reservas navegables con `DEREF`.
Lógica (Avance 2 · Semana 1)	`PKG_TURISMOVIVE`: 7 subprogramas públicos, 2 privados, 6 excepciones propias	Ejecutar cada paso del proceso de reserva con validaciones y mensajes de negocio.
Resguardo (Avance 2 · Semana 2)	`TRG_RESGUARDO_RESERVAS` (COMPOUND sobre `TABLA_RESERVAS`)	Impedir, desde la propia tabla, cualquier reserva inválida o que provoque sobreventa.

3. PKG_TURISMOVIVE: cada paso del proceso en un subprograma

Paso del proceso	Subprograma	Qué automatiza
1. Consultar el paquete	`sp_listar_actividades`, `sp_resumen_paquete`	Aplana la Nested Table con `TABLE()` y devuelve destino, número de actividades, precio y cupos (parámetros OUT).
1. Calcular la tarifa	`fn_precio_paquete`	Reutiliza `calcula_precio_total()`: (base + actividades) × (1 + comisión/100). Un solo punto de verdad.
2. Verificar cupos	`fn_cupos_disponibles`	Cupo máximo (20) − pasajeros de reservas PENDIENTE/CONFIRMADA, filtrando con `DEREF(ref_paquete)`.
3. Registrar la reserva	`sp_registrar_reserva`	Valida pasajeros, estado, viajero, paquete, fecha y cupo; obtiene `REF(v)` y `REF(p)` e inserta.
4. Confirmar / cancelar	`sp_cambiar_estado_reserva`	Normaliza y valida el estado; cancelar libera el cupo sin lógica adicional.
5. Cobrar	`fn_total_reserva`	Navega con `DEREF` al paquete y multiplica su precio por los pasajeros; detecta `REF` dangling.

Las reglas se validan **antes** del DML, el paquete no hace COMMIT (la transacción queda en manos del cliente) y los errores se publican como excepciones con nombre (`e_paquete_inexistente` −20010, `e_viajero_inexistente` −20011, `e_reserva_inexistente` −20012, `e_reserva_duplicada` −20013, `e_sin_cupo` −20014, `e_dato_invalido` −20015), de modo que la aplicación cliente puede reaccionar con `WHEN pkg_turismovive.e_sin_cupo THEN`.

4. TRG_RESGUARDO_RESERVAS: diseño y justificación

4.1 Regla de negocio protegida

«Un paquete turístico nunca puede tener más pasajeros en reservas activas (PENDIENTE o CONFIRMADA) que su cupo máximo, y toda reserva debe ser válida: pasajeros mayor a 0, estado permitido, viajero y paquete existentes y fecha de reserva anterior al inicio del viaje». El cupo máximo se toma de `pkg_turismovive.c_cupo_maximo`, así paquete y trigger comparten el mismo valor.

4.2 ¿Por qué un trigger si el paquete ya valida?

- **El paquete solo protege a quien lo usa.** Un `INSERT` o `UPDATE` directo desde SQL Developer, otra aplicación o las capas de los próximos avances lo salta. El trigger vive en la tabla y se ejecuta siempre.
- **El paquete tiene un hueco:** `sp_cambiar_estado_reserva` no revisa el cupo al reactivar una reserva CANCELADA. El trigger lo cubre (prueba 6.5 del script).
- **Integridad de las REF:** el trigger rechaza reservas con `REF` nula o dangling, el riesgo que el Avance 1 demostró con `IS DANGLING`.

4.3 Momento (BEFORE/AFTER) y nivel (ROW/STATEMENT)

Se implementó como **COMPOUND TRIGGER** `FOR INSERT OR UPDATE ON tabla_reservas`, porque la regla tiene dos partes de naturaleza distinta y cada una exige un momento y un nivel diferentes:

Sección	Reglas	Justificación
BEFORE EACH ROW	Pasajeros > 0; estado permitido; `REF` a viajero y a paquete existentes; fecha de reserva < `fecha_inicio`. Registra qué paquetes se tocaron.	**ROW** porque cada regla depende de los valores de la fila (`:NEW`). **BEFORE** porque se valida antes de escribir: una fila inválida nunca llega a la tabla y el mensaje indica exactamente qué reserva falló.
AFTER STATEMENT	Σ pasajeros activos del paquete ≤ `c_cupo_maximo` (20), una vez por paquete afectado.	**STATEMENT** porque el cupo es una regla sobre el conjunto de reservas; un trigger de fila no puede leer `TABLA_RESERVAS` mientras se modifica (ORA-04091, tabla mutante), lo que rompería los `INSERT … SELECT` del Avance 1 y todo `UPDATE`. **AFTER** porque la suma debe calcularse sobre el estado final, con todas las filas de la sentencia aplicadas; si se viola, `RAISE_APPLICATION_ERROR` revierte la sentencia completa.

DELETE no se vigila: eliminar una reserva solo libera cupo y nunca viola la regla. **Códigos compartidos:** el trigger lanza los mismos códigos del paquete (−20010, −20011, −20014, −20015); por eso el cliente captura `e_sin_cupo` por nombre sin importar quién detectó el error, y los mensajes empiezan con `[TRG_RESGUARDO_RESERVAS]` para identificar el origen.

5. Cómo el paquete y el trigger automatizan el proceso

La sección 6 del script reproduce una temporada alta en la Amazonía (paquete real 3, cupo 20, con 4 pasajeros ya reservados por Marie Dubois en la reserva real 3) y muestra la colaboración de ambos componentes:

1. Carlos Fernández reserva 16 pasajeros con `sp_registrar_reserva`: el paquete valida (16 ≤ 16 cupos) y el trigger re-valida al insertar (4 + 16 = 20). `fn_cupos_disponibles(3)` devuelve 0 en tiempo real.
2. Ana Gutiérrez intenta 1 pasajero: el **paquete** lo rechaza antes de tocar la tabla (`e_sin_cupo`).
3. Marie cancela con `sp_cambiar_estado_reserva`: se liberan 4 cupos automáticamente.
4. Ana reserva 4 pasajeros: ahora sí hay cupo y el paquete vuelve a quedar lleno.

5. Se intenta reactivar la reserva de Marie. El paquete no revisa el cupo en este paso, pero el **trigger** detecta 24 > 20 y revierte el UPDATE: se evita la sobreventa.
6. fn_total_reserva y fn_precio_paquete, junto con DEREF, generan el reporte de cobro (27 648 para la reserva de Carlos: 1728 × 16).

El paquete aporta la **automatización del flujo** (cada paso es una llamada con reglas y mensajes de negocio) y el trigger aporta la **garantía** (la regla se cumple incluso ante DML directo o un error del propio paquete).

6. Pruebas y verificación con datos reales del Avance 1

Todas las pruebas están en 03_Avance2_Semana2_TRIGGER_Y_PRUEBAS.sql y se ejecutan sobre los paquetes 1–3, viajeros 1–3 y reservas 1–3 insertados en el Avance 1. **No se crean datos de prueba aislados**: los registros nuevos (reservas 4, 5, 10–13) se apoyan en esos viajeros y paquetes reales, dentro de un SAVEPOINT que se revierte al final de cada sección. Los scripts 01, 02 y 03 se ejecutaron en **Oracle FreeSQL (Oracle Database 26ai) el 29/09/2026**: el paquete y el trigger compilaron sin errores y **todos los resultados obtenidos coincidieron con los previstos**. Evidencias: carpeta capturas/ (A: caso exitoso, B: violación bloqueada, C: nombres y datos intactos).

6.1 Subprogramas del paquete (éxito y fallo por separado)

Subprograma	Caso de éxito (resultado obtenido)	Caso de fallo (excepción capturada)
fn_precio_paquete	Paquete 3 → 1728	Paquete 99 → e_paquete_inexistente (−20010)
fn_total_reserva	Reservas 1/2/3 → 3146 / 868 / 6912	Reserva 99 → e_reserva_inexistente (−20012)
fn_cupos_disponibles	Paquetes 1/2/3 → 18 / 19 / 16	Paquete 99 → e_paquete_inexistente (−20010)
sp_listar_actividades	Paquete 2 → 3 actividades, suma 125, total 868	Paquete 99 → e_paquete_inexistente
sp_resumen_paquete	Paquete 3 → Amazonía, 1 actividad, 1728, 16 cupos	Paquete 99 → e_paquete_inexistente
sp_registrar_reserva	Ana, paquete 3, 2 pax → cupos 16 → 14	15 pax con 14 cupos → e_sin_cupo (−20014)
sp_cambiar_estado_reserva	Reserva 4 'confirmada' → CONFIRMADA	Estado 'VOLANDO' → e_dato_invalido (−20015)

6.2 Trigger: caso exitoso y violaciones deliberadas (DML directo, sin el paquete)

Prueba	Operación sobre TABLA_RESERVAS	Resultado obtenido
T1 · Éxito (captura A)	INSERT … SELECT de 16 pax en el paquete 3 (4 + 16 = 20, justo el cupo)	1 fila insertada; cupos del paquete 3 = 0
V1 · Violación (captura B)	INSERT de 1 pax más en el paquete 3 lleno (21 > 20)	ORA-20014 [TRG_RESGUARDO_RESERVAS] Sobreventa… + ORA-04088; la reserva 11 no existe
T2 / T3 · Éxito	Cancelar la reserva 10 y registrar 10 pax nuevos	Cupos 0 → 16 → 6
V2	Reactivar la reserva cancelada 10 (14 + 16 = 30)	Bloqueada, e_sin_cupo
V3	Reserva real 3 de 4 a 11 pax (10 + 11 = 21)	Bloqueada, e_sin_cupo
V4	UPDATE de las reservas reales 1, 2 y 3 en una sola sentencia (+7 pax)	Bloqueada; las 3 filas conservan 2 / 1 / 4 pax (atomicidad de sentencia)
V5 – V8	Fecha posterior al inicio, 0 pasajeros, estado 'VOLANDO', REF de viajero nula	Bloqueadas, e_dato_invalido / e_viajero_inexistente

6.3 Verificación de integración con datos reales

- **Antes de crear el trigger** (sección 1): 3 paquetes, 3 viajeros y 3 reservas; ocupación 2 / 1 / 4 ≤ 20 y ninguna reserva real viola reglas de fila (0 filas). El trigger no revisa filas existentes; por eso se comprobó primero.
- **Después de todas las pruebas** (sección 7): siguen 3 / 3 / 3 registros, las reservas 1–3 apuntan a los viajeros y paquetes 1–3 sin REF dangling, los cupos vuelven a 18 / 19 / 16, y PKG_TURISMOVIVE (PACKAGE y PACKAGE BODY) y TRG_RESGUARDO_RESERVAS están en estado VALID.

7. Verificación de nombres y lista de verificación

Los nombres se comprueban en el diccionario de datos con USER_OBJECTS, USER_TRIGGERS (tipo COMPOUND, evento INSERT OR UPDATE, tabla TABLA_RESERVAS, BEFORE_ROW = YES, AFTER_STATEMENT = YES) y USER_PROCEDURES (7 subprogramas públicos).

Punto de verificación	Evidencia	Estado
Momento y nivel del trigger diseñados y justificados por escrito	Sección 4.3; encabezado del script	☑
Trigger implementado y la regla de negocio se cumple	Script, secciones 2 y 5	☑
Cada subprograma probado con un caso de éxito y uno de fallo	Script, sección 4; tabla 6.1	☑
Trigger probado intentando violar la regla deliberadamente	Script, sección 5 (V1–V8); capturas A y B	☑
Paquete y trigger verificados sobre datos reales del Avance 1	Script, secciones 1, 6 y 7	☑
Informe final de integración (máx. 5 páginas)	Este documento	☑
Nombres PKG_TURISMOVIVE y TRG_RESGUARDO_RESERVAS iguales a la Sección 5	Script, sección 3; captura C	☑
Defensa oral ensayada: cada integrante explica un subprograma o el trigger	Tabla de responsables (abajo)	☐

Integrante	Componente que explica en la defensa
Alejandro Rojas Espinoza	TRG_RESGUARDO_RESERVAS completo: regla, BEFORE EACH ROW, AFTER STATEMENT, tabla mutante
Alejandro Yabeta Arias	sp_registrar_reserva y sp_cambiar_estado_reserva (REF, validaciones, excepciones)
Mathias Ethan Cordova Rodrigues	fn_precio_paquete, fn_total_reserva y fn_cupos_disponibles (método del tipo, DEREF)
Sebastian Maldonado Salazar	sp_listar_actividades, sp_resumen_paquete (TABLE(), OUT) y el manejo de excepciones

8. Conclusiones

El proceso de venta de TurismoVive queda automatizado de punta a punta: el paquete PKG_TURISMOVIVE convierte cada paso (consultar, tarificar, verificar cupo, reservar, confirmar/cancelar y cobrar) en una llamada con reglas y mensajes de negocio, y el trigger TRG_RESGUARDO_RESERVAS asegura, desde la propia tabla, que ningún camino (DML directo, otra aplicación o un hueco del paquete) produzca sobreventa o reservas inválidas. El diseño BEFORE EACH ROW + AFTER STATEMENT valida cada fila antes de escribirla y evalúa el cupo sobre el estado final sin caer en el error de tabla mutante. Con ello se responden los ejes de la Pregunta Rectora de tarifas dinámicas y comisiones, disponibilidad de cupos en tiempo real y paquetes flexibles, y las reservas validadas, con fecha y destino, quedan como base confiable para la analítica de demanda estacional de los próximos avances.